



Unsupervised Steering Vectors from Learned Classifiers: A Summary

October 5, 2026

WHAT THE METHOD DOES

The method, which we call an *Ontologizer*, learns a set of small classifiers that together describe a neural network’s internal representation. Each classifier sorts every input into one of k categories (we use $k = 32$), and each category stores one vector. An input is described by which category each classifier chose, and it is reconstructed as the sum of the chosen vectors. Nothing is labelled: the classifiers and their vectors are learned only by asking the sum to reproduce the representation as closely as possible.

Because each classifier’s categories are mutually exclusive, changing one classifier’s answer for an input — from category a to category b — swaps one stored vector for another. That swap is a steering intervention: it moves the representation from “ a ” to “ b ” along a single, named axis, and the model supplies the direction.

THE PROBLEM IT ADDRESSES

To steer a model is to change its internal state so that it behaves differently in a chosen way. The most reliable current method is a *steering vector*: take the average representation of examples with a property, subtract the average of examples without it, and add the difference to new inputs [1, 2]. This works well, but someone must decide which property to look for and supply labelled examples of both sides, so it can only find what its user already suspects is there.

Sparse autoencoders (SAEs) find features without labels: each input is described by a few active features out of thousands [3, 4]. But their features switch on and off independently, and adding one to an input tends to change others and to damage the input in unpredictable ways; on standard steering benchmarks, simple supervised baselines beat them [5]. The goal here is steering vectors that need no

labels but keep the useful shape of the supervised kind: each one a choice between alternatives on a shared axis.

WHAT IT ASSUMES, AND WHY

1. **Concepts are directions that add.** A representation is, approximately, a sum of vectors for the things it encodes. This is the linear representation hypothesis [6], and it is what makes supervised steering vectors work at all.
2. **Much of the variation is categorical.** A sentence has one dominant topic, one language, one register; it is more natural to describe it by choosing among alternatives than by switching many independent features on and off. We check this rather than assume it: the clearest classifier the method learns on sentence embeddings sorts by topic and genre (crime news, legislation, encyclopedic description, and so on).
3. **The categories are a property of the data, not of the training run.** If the method recovers real structure, a second run from a different random start should find the same classifiers. This is the assumption that has *not* held, and it is the main open problem (below).

HOW IT WORKS, INTUITIVELY

Think of describing a sentence by answering a fixed list of multiple-choice questions, where the questions and answers are learned. The first round of questions captures the broad outline; the model then measures what that outline gets wrong and asks a second round about the remaining error, and so on for a few rounds. The answers can be soft (a mixture of categories) or hard (exactly one category per question). In the hard version an input is stored as a short list of discrete choices — 1900 bits for a sentence embedding — with no continuous numbers at all.

We apply it to two representations: SONAR sentence embeddings [7], which can be decoded back into text, so a steered embedding can be read; and the internal activations of GPT-2 small [8], where the reconstruction can be put back into the model to see how much of its behaviour survives.

HOW IT COMPARES WITH EXISTING METHODS

Against supervised steering — the comparison that matters. Two tests so far. First, steering sentence embeddings toward a target language: forcing one of the method’s learned classifiers moved the decoded text toward that language 0.43 times as far as the supervised difference-of-means vector did, and 5 times further than a linear-probe direction. The method never saw a label in training; labels were used only to pick which category to force. All methods, the supervised one

included, damaged unrelated content about as much as a random step of the same size.

Second, a direct test of whether the method’s own steering direction is a good way to reach its own categories. For each category we also fitted the two standard supervised vectors — difference of means and a linear probe — using the method’s own assignments as labels, and gave every direction the same step size. This is a generous baseline, since it already knows which inputs belong to the category. For the soft version of the method, its own direction reached the category on 75% of inputs, against 46–47% for the supervised vectors. For the hard multi-round version, its own direction reached only 37%, against 91–94%: its categories can be reached, but its own direction does not point at them. Nearly every method, supervised or not, disturbed the other classifiers about as much as a random step of the same size.

So the method recovers useful steering directions without labels, and in its soft form they are better than supervised vectors at reaching its own categories. It has *not* been shown to beat supervised steering of an externally defined concept; the one such test puts it at 0.43 of the supervised effect.

Against SAEs on steering. Measured directly in embedding space, at each model’s own natural step size, a feature of a standard SAE (32 active features per input) takes effect on 46–50% of inputs, and other SAE settings range from 8% to 82%; the method’s multi-round version takes effect on only 8–19%, while disturbing other classifiers as much as a random step would. A simplified one-round version of the method does best of all on side effects: it takes effect on 41% of inputs while disturbing only 1.4% of the other classifiers, against 18% for a random step.

Against SAEs on reconstruction (a secondary check). Describing a sentence embedding with 1900 bits of discrete choices, the method leaves 15% of the variance unexplained, against 26% for an SAE using a similar number of bits plus 160 continuous coefficients. On GPT-2, an 800-bit code preserves 97.6% of the model’s performance, against 96.2% for an SAE with similar error.

WHAT WE CAN CLAIM

On dense sentence embeddings, the method describes each embedding with a small set of discrete classifier choices far more efficiently than a sparse autoencoder of matched code size describes it, and at least one of those classifiers — topic and genre — is found again in every training run and can be described well enough for a blind judge to recognise its categories. Whether its classifiers are in general easier to interpret than SAE features is not established: descriptions of individual categories are no more accurate than descriptions of SAE features once the codes’ density is accounted for, and most classifiers do not reappear across training runs. Testing the stronger claim needs labelled factors (such as topic) to compare against, classifiers that reproduce, and descriptions scored at matched code size.

WHAT DOES NOT WORK YET

- **The classifiers are not reproducible.** Two runs from different random starts reconstruct equally well, but agree on almost none of their classifiers beyond the first round; only the topic classifier reappears reliably. A classifier that is different on every run cannot be named, so this limits everything else.
- **Steering through the input is not targeted.** Pushing an input toward one category changes other classifiers' answers about as much as a random push of the same size, and the multi-round structure doubles that.
- **The stored vectors cannot be read on their own.** Decoding a category's vector into text does not describe what the category selects; examples of inputs are needed.

NEXT

The test the method still needs is steering toward externally defined concepts — topic, language, register — with supervised vectors fitted to those concepts' labels, SAE features and the method's own categories scored on the same targets, success measure and step size. The same comparison on the method's own categories is done (above). In parallel, the reproducibility problem has to be solved, or the reproducible classifiers identified, before any classifier is given a name.

REFERENCES

- [1] Alexander Matt Turner et al. *Activation Addition: Steering Language Models Without Optimization*. 2023. arXiv: 2308.10248 [cs.CL]. URL: <https://arxiv.org/abs/2308.10248>.
- [2] Nina Rimsky et al. *Steering Llama 2 via Contrastive Activation Addition*. 2024. arXiv: 2312.06681 [cs.CL]. URL: <https://arxiv.org/abs/2312.06681>.
- [3] Trenton Bricken et al. "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning". In: *Transformer Circuits Thread* (2023). URL: <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [4] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: <https://arxiv.org/abs/2406.04093>.
- [5] Zhengxuan Wu et al. *AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders*. 2025. arXiv: 2501.17148 [cs.CL]. URL: <https://arxiv.org/abs/2501.17148>.

- [6] Kiho Park, Yo Joong Choe, and Victor Veitch. *The Linear Representation Hypothesis and the Geometry of Large Language Models*. 2023. arXiv: 2311.03658 [cs.CL]. URL: <https://arxiv.org/abs/2311.03658>.
- [7] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: <https://arxiv.org/abs/2308.11466>.
- [8] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.